

MistleTunnel: Attacking KASLR on Apple M-Series Silicon with a Novel TLB Side Channel

Abstract

KASLR is a critical defense on macOS, whose implementation on M-series silicon and the latest macOS is further strengthened by hardware–software mitigations, with no practical bypass. We perform a systematic reverse-engineering of the TLB on M-series silicon and uncover a novel user–kernel TLB side-channel, TLBarge, that enables precise monitoring of kernel instruction execution. Leveraging TLBarge, we demonstrate MistleTunnel, the first attack that compromises macOS 26’s 15-bit KASLR by recovering its lower 8 bits. Operating solely through normal syscalls, MistleTunnel collapses the effective KASLR entropy to 0.39%, significantly reducing its intended security.

1 Introduction

Kernel Address Space Layout Randomization (KASLR) protects against kernel address disclosure by randomizing the kernel base address, thereby mitigating a wide range of follow-up memory corruption attacks [6, 22]. Nevertheless, numerous studies have shown that KASLR can be compromised with hardware side-channel attacks [4, 7, 9, 11, 13, 14, 16, 17, 19–21, 25]. A typical KASLR attack involves accessing a kernel address and analyzing the impact of the access through hardware side channels. However, direct access to a kernel address from user space is recognized as an illegal behavior by the operating system (OS). To bypass this, these attacks either avoid triggering exceptions by leveraging various CPU features [4, 7, 9, 13, 14, 16, 19–21, 25], or exploit timing side channels caused by flaws in the OS or CPU’s handling of page faults or privilege check failures [11, 17, 23].

However, Apple has introduced additional protections for kernel page-table entries (PTEs) and address-space management in M-series silicon, including Double Mapping [24] and the Secure Page Table Monitor (SPTM) [2], which effectively mitigate prior attacks and significantly raise the difficulty of mounting KASLR attacks on this platform.

To date, the only known KASLR attack targeting M-series silicon is SysBumps [13], which abuses transient execution to speculatively access attacker-crafted kernel addresses and detect mapping status through microarchitectural leakage. However, such speculative access still depends on specific software gadgets and can be easily mitigated through software patches with minimal overhead. Indeed, this vulnerability was patched in macOS 15.2 [1]. As a result, prior to our work, no KASLR attacks had been demonstrated on macOS 15.2 or later.

In this work, we observe that once the KASLR slide is fixed, the TLB accesses induced by certain syscalls on Apple Silicon follow a stable and deterministic pattern. Motivated by this insight, we propose a novel TLB-based KASLR attack, named MistleTunnel, that exploits an overlooked vector beyond the reach of existing defenses on the latest macOS: the TLB traces generated by normal

syscall execution. Since syscalls inherently invoke specific kernel functions, their associated instruction fetches constitute entirely legal and unavoidable kernel accesses from the OS’s perspective. By correlating syscall-specific TLB access patterns with the KASLR slides and monitoring these traces from user mode, we can reliably compromise KASLR.

However, inferring the KASLR slide from syscall TLB traces poses significant challenges. To recover the addresses of executed kernel instructions, we must determine precisely which TLB sets are accessed during a syscall. Therefore, constructing this mapping requires set-granularity visibility into instruction execution on the TLB. Furthermore, in the attack phase, all probing must be performed from non-privileged user mode, making the problem even more difficult, as it rules out techniques that rely on elevated permissions. These two requirements necessitate a user-privileged, set-granularity side-channel primitive to monitor kernel instruction execution. However, to date, the most fine-grained probing of instruction entries in the TLB requires root privileges and cannot achieve set-granularity [13].

To address these limitations, in this work, we perform a systematic reverse-engineering of TLB architectures on M-series silicon, uncovering previously unaddressed characteristics, including shared L2 TLB, load behavior, and eviction behaviors. Leveraging these features, we construct a user-privileged set-granularity side-channel primitive, named TLBarge, for monitoring kernel instruction execution. Based on TLBarge, we successfully analyzed the mapping between syscall TLB traces and KASLR slides and demonstrated a confused-deputy attack against KASLR. By invoking a syscall as an unprivileged user and analyzing the syscall trace, we break the lower 8 bits of the 15-bit KASLR in macOS 26, reducing its entropy to 0.39% and significantly weakening its intended security, bypassing existing mitigations.

In summary, this paper makes the following contributions:

- We conduct a systematic reverse-engineering of TLB properties on Apple M-series silicon, unveiling its shared nature, load behavior, and eviction behaviors.
- We introduce TLBarge, a novel user-privileged, set-granularity side-channel primitive that enables monitoring of kernel instruction execution traces via the TLB.
- We demonstrate the first real-world side-channel attack that compromises the lower 8 bits of macOS 26’s 15-bit KASLR, bypassing all existing mitigations on modern Apple Silicon.

2 Related Work

KASLR attacks based on hardware side channels inevitably involve accessing kernel addresses and analyzing the results through side channels, while direct access to kernel addresses from user space is recognized as an illegal behavior from the OS’s perspective. To the best of our knowledge, current KASLR attacks can be categorized into two types:

Utilizing Timing Side Channels from Illegal Behavior Handling. These attacks do not avoid triggering exceptions or privilege check failures when accessing kernel addresses from user space, but exploit timing side channels caused by flaws in the OS or CPU's handling mechanism for illegal behavior. Double page fault [11] and Seddigh et al. [23] exploit the time difference between two page faults triggered by the same kernel address to infer whether the address has been allocated. TagBleed [17] measures the abort time of an invalid syscall to infer part of the KASLR.

Bypassing Illegal Behavior Detection. These attacks leverage various CPU features to avoid exceptions caused by accessing kernel addresses from user space, and use microarchitectural side channels to determine whether the accessed address is mapped. Gruss et al. [9], EntryBleed [21], and Lipp et al. [19] exploit the fact that x86 prefetch instructions do not trigger exceptions when accessing kernel addresses. Drk [14] and PLATYPUS [20] use Intel Transactional Synchronization Extensions (TSX) to suppress exceptions caused by invalid or inaccessible memory accesses. Similarly, AVX-TSCHA [16] uses x86 Advanced Vector Extensions (AVX) to suppress exceptions when accessing kernel addresses. Evtvushkin et al. [7], Fallout [4], FlushConflict [25], and SysBumps [13] rely on transient execution triggered by various prediction mechanisms to speculatively access kernel addresses.

Comparison to Previous Work. The attacks discussed above all rely on illegal behaviors recognized by the OS and can be mitigated through software updates [1, 5, 12, 15, 18]. In contrast, our attack, MistleTunnel, performs a confused-deputy attack by legally invoking a syscall to trigger inevitable TLB loads of kernel instruction pages, which is entirely legitimate from the OS's perspective. We construct a mapping between syscall TLB traces and KASLR bits, and compromise KASLR with the TLB traces of a legally invoked syscall. Notably, MistleTunnel remains effective on macOS 26, even with the SysBumps mitigation in place, and is, to the best of our knowledge, the only available KASLR attack targeting the latest macOS and M-series silicon.

3 Characterizing TLBs on M-Series Silicon

In this section, we present the reverse-engineering of the essential characteristics needed to construct a set-granularity side-channel for monitoring the TLB state. We summarize the results in Table 2.

Platforms. We conducted our experiments on Apple M2-M4 series CPUs, running macOS 26, with a default page size of 16 KB, covering both P-cores and E-cores. We implemented monitoring of TLB-related events listed in Table 1 using an open-source project based on KPC [26]. We utilize the macOS thread affinity API to pin our experiments to P-cores and E-cores, respectively.

3.1 Set Associativity and Mapping

We begin by determining the set associativity and mapping function of the TLBs to enable the construction of eviction sets. SysBumps [13] has confirmed that the TLBs on M-series silicon follow set-associative structures and employ *linear index mapping* [8] on M1-M2 Apple silicon. Specifically, for a TLB with s sets and w ways, the target set of a virtual page address $page_{VA}$ is computed as $(page_{VA} \gg 14) \bmod s$, where the lower 14 bits of the virtual

Table 1: TLB-related KPC events.

Event	Description
L1I_TLB_MISS_DEMAND	Demand fetch misses that require a new cache line fill of L1 iCache.
L1D_TLB_MISS	Load & store misses in L1 dTLB.
L2_TLB_MISS_INSTRUCTION	Instruction fetch misses in L2 TLB.
L2_TLB_MISS_DATA	Load & store misses in L2 TLB.

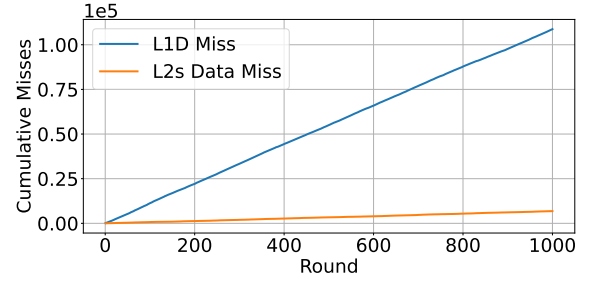


Figure 1: Load behavior and shared TLB experiment results on M3 P-Core.

address encode the in-page offset and are therefore excluded from TLB indexing. Leveraging this property, we infer the set and way sizes of the TLBs by constructing eviction sets under hypothesized configurations and examining the resulting TLB miss patterns.

3.2 Shared TLB and Load Behavior

In this section, we confirm the following characteristics:

L1 Load Fills L2: Both L1 and L2 TLBs are populated when a new address is translated.

Shared L2 TLB: The L2 TLB is a shared structure for both instructions and data; the entries loaded into the L2 TLB are not distinguished as data or instruction entries.

To validate these characteristics, we designed the following experiment:

- (1) Prepare a set of pages with both executable and readable permissions as the probe set.
- (2) Trigger L1 iTLB loads by executing the probe set pages.
- (3) Trigger dTLB loads by reading the probe set pages and observe the number of L1D_TLB_MISS and L2_TLB_MISS_DATA events.

We conducted multiple experiments and collected data on the L1D_TLB_MISS and L2_TLB_MISS_DATA events, observing trends over time. The experimental results, shown in Figure 1, reveal a significant number of misses in the L1 dTLB, as expected, since an iTLB load does not trigger a dTLB load. However, the number of TLB misses in the L2 TLB due to data access is exceedingly small. This suggests that the L1 iTLB load triggers an L2 TLB load, and the entries loaded into the L2 TLB via the iTLB load can also be used for data access. We observe the same trend across all M2-M4 series CPUs, validating our assumptions. We then refer to L2 TLB as the L2 sTLB (shared TLB) henceforth.

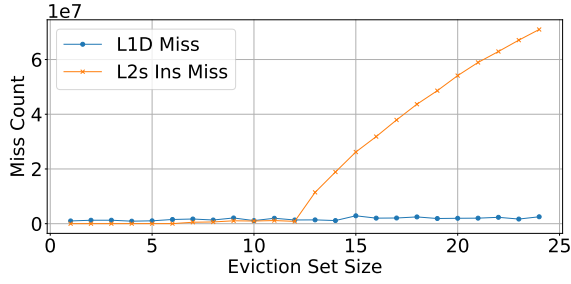


Figure 2: Confirmation of eviction behavior (2) on M3 P-Core.

3.3 Eviction Behavior

In this section, we further analyze whether L1 TLB and L2 sTLB affect each other during evictions and confirm the following two **eviction behaviors**:

① **L1 Eviction does not affect L2**: Entries evicted from L1 TLB are not loaded into the L2 sTLB.

② **L2 Eviction does not affect L1**: Entries evicted from L2 sTLB do not trigger eviction of corresponding entries in L1 TLB.

Validation of ①. We conducted the following experiment:

- (1) Prepare readable pages as the probe set.
- (2) Trigger L1 dTLB loads and L2 sTLB loads by reading the probe set pages.
- (3) Evict L2 sTLB entries of the probe set via instruction fetches.
- (4) Evict the L1 dTLB entries of the probe set using data accesses.
- (5) Trigger dTLB loads and observe the number of L1D_TLB_MISS and L2_TLB_MISS_DATA events.

We observe that the number of L1D_TLB_MISS events is nearly identical to that of L2_TLB_MISS_DATA events. This indicates the entries evicted from L1 dTLB are not reinserted into L2 sTLB.

Validation of ②. We conducted the following experiment: We prepared a data probe set P and an instruction eviction set E , both mapped to the same L2 set, allowing E to evict the L2 sTLB entries of P . We varied the eviction set size and applied the method described in Algorithm 1 to observe the impact of L2 sTLB eviction on the L1 dTLB. The experimental results are shown in Figure 2. When the eviction set size reached the L2 sTLB set size, we observed a significant increase in L2 sTLB instruction misses, indicating that E occupied the entire L2 sTLB and began to self-evict, suggesting the entries loaded into the L2 sTLB by P were fully evicted. However, no significant increase in L1 dTLB misses was observed, suggesting that eviction of a specific entry in L2 does not trigger the eviction of the corresponding entry in L1. This behavior was consistently observed across all M2-M4 Series CPUs.

4 Build User-Kernel iTLB Side-Channel with Set-Granularity

In this section, based on the findings of **Load Behavior**, **Eviction Behaviors**, and **Shared L2 TLB**, we develop two set-granularity side-channel primitives, named **TLBarge-L1i** and **TLBarge-L2s**, to monitor the state of L1 iTLB and L2 sTLB separately. Instead of inferring the iTLB state by directly measuring the latency of instruction execution, which is highly unstable due to factors such as branch prediction and prefetching, we leverage the shared nature

Algorithm 1: Validation of eviction behavior (2)

```

Input : eviction_set_size, page_base, l2s_set_size, l1d_way_size
         set_num (set_num < min(l1d_set_size, l1i_set_size))
Output: (l1d_miss_count, l2s_ins_miss_count)
1 Function calc_conflict_addr (page_base,
   set_num, way_seq):
2   return page_base + set_num * page_size
   + way_seq * l2s_set_size * page_size
3 for way_seq ← 1 to l1d_way_size do
4   load(calc_conflict_addr(page_base, set_num, way_seq));
5 read_pmu(l1d_miss_count_start, l2s_ins_miss_count_start);
6 for way_seq ← l1d_way_size + 1
   to l1d_way_size + eviction_set_size do
7   execute(calc_conflict_addr(page_base, set_num, way_seq));
8 for way_seq ← 1 to l1d_way_size do
9   load(calc_conflict_addr(page_base, set_num, way_seq));
10 read_pmu(l1d_miss_count_end, l2s_ins_miss_count_end);
11 l1d_miss_count ←
   l1d_miss_count_end - l1d_miss_count_start;
12 l2s_ins_miss_count ←
   l2s_ins_miss_count_end - l2s_ins_miss_count_start;

```

Table 2: Properties of TLB on M-Series Silicon

TLB Property	M2 Pro P-core	M2 Pro E-core	M3 P-core	M3 E-core	M4 P-core	M4 E-core
L1 Load Fills L2	✓	✓	✓	✓	✓	✓
Shared L2 TLB	✓	✓	✓	✓	✓	✓
L1 Eviction Affects L2	✗	✗	✗	✗	✗	✗
L2 Eviction Affects L1	✗	✗	✗	✗	✗	✗
L1 dTLB						
Number of sets	64	32	32	32	32	32
Number of ways	4	6	5	6	5	6
Hash function	Linear	Linear	Linear	Linear	Linear	Linear
L1 iTLB						
Number of sets	32	32	32	32	32	32
Number of ways	6	6	6	6	6	6
Hash function	Linear	Linear	Linear	Linear	Linear	Linear
L2 sTLB						
Number of sets	256	256	256	256	256	256
Number of ways	12	8	8	12	12	8
Hash function	Linear	Linear	Linear	Linear	Linear	Linear

of instruction and data entries in the L2 sTLB. By converting the states of instruction TLB entries in either the L1 iTLB or L2 sTLB to data entry states in the L2 sTLB and probing with data loads, we obtain more reliable results. We validate the effectiveness of TLBarge in monitoring kernel instruction execution and confirm that TLBarge can clearly distinguish even a single access to the TLB by kernel instructions.

Figure 3 shows how our primitive works. We use “—” to represent entries of no interest, “I” to denote instruction entries, and “D” for data entries.

4.1 TLBarge-L1i

As shown in Figure 3a, TLBarge-L1i consists of two phases: the Prime phase (123) and the Probe phase (45). As shown in Table 2, the L2s set size is a multiple of the L1i set size, resulting in a one-to-many mapping between L1i and L2s loads. However,

leveraging the characteristics revealed in Section 3, we can convert an L1i iTLB state update into a timing change in a determined L2s set. We ensure that I and D map to the same L2 sTLB set in Figure 3a, as the addresses of I and D are controllable by us.

Prime Phase: ① Prime the L1 iTLB via instruction fetch with I , also causing the load of I to L2 sTLB; ② Evict the corresponding L2 sTLB entries of the primed I with data load D . ③ Evict the L1 dTLB entry of D to ensure that subsequent data loads will not hit L1 dTLB and reflect the states of L2 sTLB entry of D .

Probe Phase: In the probing phase, two scenarios might occur: (1) kernel access occurs and (2) no kernel access. In terms of the first scenario, the primed I is evicted by kernel iTLB load, ④ a subsequent access to I will cause it to be reloaded into both the L1 iTLB and the L2 sTLB, thereby evicting the D entry. Consequently, in ⑤ when probing D , higher access latency will occur because of TLB miss. For the second scenario, I remains in the L1 iTLB, ④ access to I will result in an L1 iTLB hit, which will not affect the L2 sTLB state. Thus, ⑤ probing D will still yield L2 sTLB hits, leading to lower latency.

4.2 TLBarge-L2s

We directly monitor the corresponding L2sTLB load of the L1 iTLB load in this primitive. We ensure that D maps to the L2 sTLB set in Figure 3a, as the address of D is controllable by us.

Prime Phase: ① Prime the L2 dTLB via data load with D ; ② Evict the corresponding L1 dTLB entry of the primed D to ensure that subsequent data loads will not hit L1 dTLB and reflect the states of the L2 sTLB entry of D .

Probe Phase: If the kernel access occurs, the primed D is evicted by the kernel L2 sTLB load, and probing D in ③ will result in a TLB miss, leading to higher access latency. Otherwise, D remains in the L2 sTLB, and probing D in ③ will result in an L2 sTLB hit, leading to lower latency.

4.3 Validation of TLBarge in Detecting Kernel Instruction Execution

We validate the effectiveness of our primitives in monitoring kernel instruction execution on a Mac mini with Apple M4 silicon, running macOS Tahoe 26.0.

We define a function `foo()` within a macOS Kernel Extension (KEXT), obtain its address to calculate the corresponding TLB set mapping, and implement a kernel `sysctl` handler that controls whether `foo()` is invoked based on request parameters.

We controlled the invocation of the kernel function `foo()` through the `sysctl` handler between the prime and probe phases, and compared the probe latency differences in both cases. The results are shown in Figure 4a and Figure 4b. A single invocation of the kernel function causes a significant increase in latency. It confirms that an unprivileged user can monitor the trace of kernel function execution using our set-granularity side-channel TLBarge.

5 Implementation of MistleTunnel

In this section, we explain our methodology for leaking KASLR through normal syscall execution and TLB traces, and then demonstrate how the leaked KASLR information makes memory corruption attacks feasible.

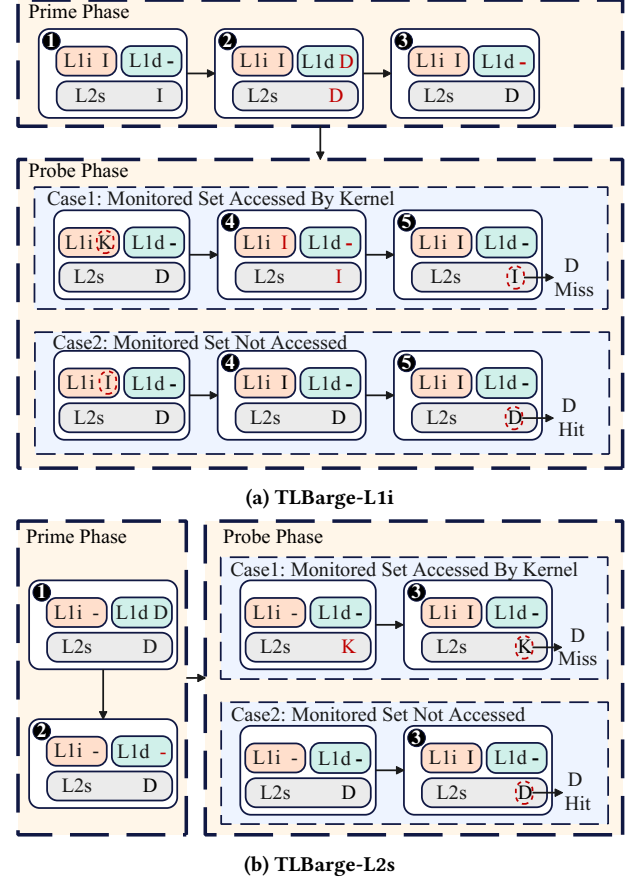


Figure 3: Monitoring of Kernel Instruction Execution with Set-Granularity

Threat Model: In this attack, we assume an unprivileged attacker who can execute arbitrary code in a user process but has no privileges over the operating system. The attack is assumed to be carried out on macOS, with patches against previous KASLR attacks and protected by SIP [3], Double Mapping [24] and SPTM [2]. The attacker does not have read access to the timestamp register. To achieve high-resolution timing measurements, the attacker employs a counting thread [10]. The attacker wants to carry out memory corruption attacks [6, 22] which require to derandomize the kernel base and the runtime addresses of target victim kernel functions.

5.1 Methodology for KASLR Leakage via TLB

Based on the analysis of the Apple KASLR implementation in previous work, the Kernel Base is selected from 2^{15} slots aligned to 16 KiB [13]. In other words, as shown in Figure 5, for any function in the kernel, the bits [28:14] of its instruction virtual address are affected by KASLR. According to our analysis of TLB set-associativity on the M4 P-Core, the L1i TLB and L2 sTLB use bits [18:14] and [21:14], respectively, which overlap with the KASLR-affected bits, making these overlapping bits vulnerable to exploitation through

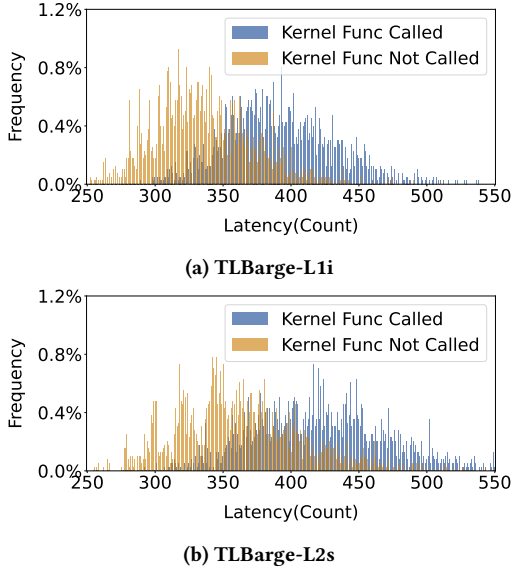


Figure 4: Both TLBarge-L1i and TLBarge-L2s can distinguish a single kernel function execution

TLB traces. Furthermore, since the L2 sTLB uses more bits for indexing (8 bits) compared to the L1i TLB (5 bits), the trace on the L2 sTLB can reveal more KASLR bits. Specifically, we denote the KASLR slide divided by 16 KiB as *slot*. If the static address of a kernel function is denoted by *fixed_addr*, the actual address of the function in the kernel is $fixed_addr + (slot \ll 14)$. Its set selection in the L2 sTLB is given by:

$$\begin{aligned} set &= ((fixed_addr + (slot \ll 14)) \gg 14) \bmod 256 \\ &= (slot + (fixed_addr \gg 14)) \bmod 256 \end{aligned}$$

Thus, the accessed sets in the L2 sTLB exhibit a linear relationship with *slot*, with a slope of 1 and offsets determined by the fixed addresses of the functions in the execution path. The offsets of the instruction pages in the execution path can be analyzed through multiple reboots, as we will detail in Section 5.2. Additionally, we can determine the accessed sets by measuring their latency with TLBarge-L2s. If we denote the lower 8 bits of *slot* as *slot₈*, after determining the other two variables, *slot₈* can then be derived by:

$$slot_8 = (set - (fixed_addr \gg 14)) \bmod 256$$

Identifying a Unique Correct *slot₈* with Featured Offsets. During the recovery of the *slot₈* through TLBarge, multiple sets may show positive access results due to system noise or limited timing resolution, leading to multiple possible guesses for *slot₈*. To solve this problem, we can identify multiple offsets derived from the *fixed_addr* of each instruction page in the execution path, as function calls are highly likely to span across pages. We refer to these offsets as **featured offsets**. During the attack phase, we search for a unique *slot₈* such that all sets whose indices equal the sum of *slot₈* and the corresponding featured offset (mod 256) show high probing latency. This significantly reduces the chance of false positives, as shown in Section 6.2, with this approach, we can achieve 100% accuracy in just a few seconds.

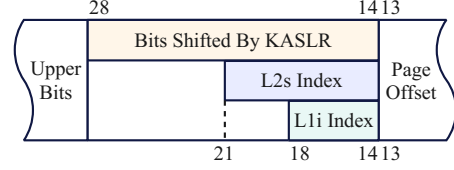


Figure 5: Bits used in L1i and L2s TLB indexing on M4 P-Core and their overlay with KASLR-affected bits

5.2 Identifying Featured Offsets of the Execution Path

As we mentioned above, we need to choose a syscall and get its featured offsets for leaking the KASLR bits. A syscall is particularly suitable for our attack if its execution path remains fixed across multiple repetitions. We analyzed multiple syscalls provided by macOS and chose `getpid()` as our target syscall function. We performed multiple reboots on macOS Tahoe 26.0 across M2–M4 M-series silicon, and implemented a macOS KEXT to retrieve the current *slot₈* value. We then monitored the syscall L2 sTLB traces using TLBarge-L2s. The results for the M4 are shown in Figure 6, which align with our analysis in Section 5.1. The sets with high access latency (indicating frequent access) exhibit a linear relationship with *slot₈* and a slope of 1. Additionally, we observe multiple distinct featured offsets that consistently appear, confirming the inference of multiple featured offsets discussed in Section 5.1.

5.3 Leaking KASLR Bits

With the featured offsets we get in Section 5.2, the KASLR bits leakage process is shown as follows:

Step 1: The attacker primes the monitored set with TLBarge-L2s to initialize the state of the monitored set, and creates a counting thread for latency measurement.

Step 2: The attacker invokes the target syscall to trigger accesses to specific sets in TLB, leaking the information of the current *slot₈*.

Step 3: The attacker probes the monitored set with TLBarge-L2s, measuring the access latency using the counting thread, to identify if the syscall access trace drops on the monitored set.

Step 4: The attacker repeats Step 1 to 3 for all sets of the L2 sTLB (0 to 255), records the probing latency for each set, sorts the sets by probing latency, and identifies a *slot₈* such that the sum of *slot₈* and all featured offsets results in sets that show high access latency.

Step 5: Once *slot₈* is determined, the attacker can begin derandomizing the runtime address of the target kernel function. The attacker obtains the symbol table information and the starting address of the `__TEXT_EXEC` segment to calculate the *fixed_addr* of the target function. Both pieces of information can be easily retrieved by a non-privileged user using the `nm` and `kernutil` tools to inspect the kernel image and the kernel cache file. The corresponding runtime address, *runtime_addr*, is then given by:

$$runtime_addr = fixed_addr + (slot \ll 14)$$

Having successfully derandomized the lower 8 bits of *slot* using MistleTunnel, the entropy of the KASLR address offset is reduced to 0.39%, leaving only 7 bits unknown, as shown in Figure 7. If the attacker intends to launch a memory corruption attack targeting

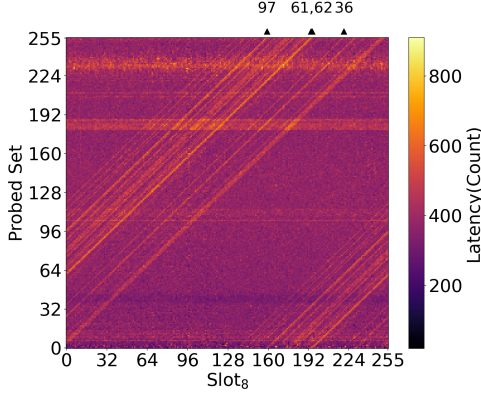


Figure 6: Heatmap of set probing latency after invoking `getpid()` on m4 Mac mini running macOS Tahoe 26.0, the featured offsets are 36,61,62,97

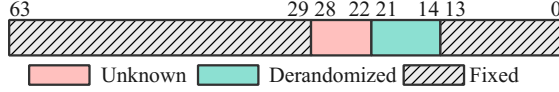


Figure 7: Only 7 bits remain unknown after MistleTunnel

kernel space [6, 22], which requires the full address of a targeted kernel function, the search space for the correct full address is reduced to only 128 possibilities, making a brute-force attack feasible.

6 Evaluation

In this section, we first evaluate the efficiency of our side-channel TLBarge. Then we evaluate the attack performance of MistleTunnel and compare it with the state-of-the-art attack, SysBumps, demonstrating our effectiveness under the latest mitigation.

Experiment Setup. We perform evaluation on M2-M4 silicon chips running macOS 26.0. The correct `slot8` values are obtained using macOS KEXT to measure the accuracy of MistleTunnel.

6.1 Evaluating the Efficiency of TLBarge

To evaluate the efficiency and discriminatory power of our two primitives, we measure the time cost for the prime and probe phases, as well as the latency gap between monitored set access and idle states. The results are shown in Table 3, the time costs for all phases are in the nanosecond range, and the latency difference can be clearly distinguished by averaging results from multiple experiments.

6.2 Evaluation of MistleTunnel

As inferred in Section 5.1, and as shown in Table 4, by using all featured offsets as constraints, we can always uniquely determine a `slot8`. As demonstrated in Table 5, our attack on M2-M4 chips achieves 100% accuracy in just a few seconds. We further compare our attack with the only reported KASLR attack targeting Apple Silicon, SysBumps [13]. After the mitigation against SysBumps was

Table 3: Evaluation of TLBarge

Primitive	CPU	Prime time cost (ns)	Accessed probing latency (ns)	Idle probing latency (ns)
TLBarge-L1i	M2 pro	5678.79	68.03	55.85
	M3	4497.29	95.12	80.49
	M4	4406.42	93.72	79.44
TLBarge-L2s	M2 pro	305.25	109.14	94.77
	M3	300.12	99.13	82.97
	M4	298	98.21	82.2

Table 4: Only the Correct `slot8` can satisfy the constraints of all the 4 featured offsets of 36,61,62,97 on M4.

Correct <code>Slot8</code>	Top-15 L2 sTLB Sets (Sorted By Probing Latency)															
102	112	171	92	180	16	109	138	129	210	197	163	182	199	164	196	
107	174	60	168	181	202	143	169	188	163	171	221	7	204	133	114	
108	91	232	170	169	205	144	245	229	119	174	99	183	22	81	7	
111	172	233	9	207	208	114	236	181	105	80	173	147	225	116	166	
132	241	194	193	210	238	99	183	12	184	207	168	35	229	187	232	
146	243	227	233	242	200	107	111	241	207	158	206	208	18	182	212	

Table 5: Comparison of Existing KASLR-Breaking Attacks on M-Series Silicon

Attack	CPU	OS	Leaked Bits	Time (s)	Accuracy
SysBumps	M2 Pro	15.1-	15	2.21	96.3%
	M2 Pro	15.2+	✗	✗	✗
MistleTunnel	M2 Pro	26	8	7.99	100%
	M3	26	8	6.01	100%
	M4	26	8	5.85	100%

applied, the SysBumps attack failed to execute on macOS 15.2+, while MistleTunnel still successfully executes on macOS 26.0.

7 Conclusion

In this paper, we present MistleTunnel, a confused-deputy attack exploiting the normal execution of syscalls to leak KASLR bits on Apple M-series silicon, bypassing existing mitigations in macOS. To differentiate syscall TLB traces, we uncover key characteristics of the TLB architecture on M-series silicon to develop TLBarge, a user-privileged, set-granularity TLB side-channel for monitoring kernel instruction execution. On the latest macOS 26.0, despite patches against KASLR attacks, our attack successfully recovers 8 bits out of 15, reducing the entropy to 0.39% and making brute-force attacks exploiting memory corruption vulnerabilities feasible.

References

- [1] Apple. 2024. About the security content of macOS Sequoia 15.2. <https://support.apple.com/en-us/121839>. Accessed: August 2025.
- [2] Apple. 2025. Operating system integrity. <https://support.apple.com/en-hk/guide/security/sec8b776536b/web>. Accessed: August 2025.
- [3] Apple. 2025. System Integrity Protection. <https://developer.apple.com/documentation/security/disabling-and-enabling-system-integrity-protection>. Accessed: August 2025.
- [4] Claudio Canella, Daniel Genkin, Lukas Giner, Daniel Gruss, Moritz Lipp, Marina Minkin, Daniel Moghimi, Frank Piessens, Michael Schwarz, Berk Sunar,

- Jo Van Bulck, and Yuval Yarom. 2019. Fallout: Leaking Data on Meltdown-resistant CPUs. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. ACM, London United Kingdom, 769–784. <https://doi.org/10.1145/3319535.3363219>
- [5] Claudio Canella, Michael Schwarz, Martin Haubenwallner, Martin Schwarzl, and Daniel Gruss. 2020. KASLR: Break it, fix it, repeat. In *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security*. 481–493.
- [6] Mauro Conti, Stephen Crane, Lucas Davi, Michael Franz, Per Larsen, Marco Negro, Christopher Liebchen, Mohaned Qunaibit, and Ahmad-Reza Sadeghi. 2015. Losing control: On the effectiveness of control-flow integrity under stack attacks. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. 952–963.
- [7] Dmitry Evtushkin, Dmitry Ponomarev, and Nael Abu-Ghazaleh. 2016. Jump over ASLR: Attacking Branch Predictors to Bypass ASLR. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, Taipei, Taiwan, 1–13. <https://doi.org/10.1109/MICRO.2016.7783743>
- [8] Ben Gras, Kaveh Razavi, Herbert Bos, and Cristiano Giuffrida. 2018. Translation leak-aside buffer: Defeating cache side-channel protections with {TLB} attacks. In *27th USENIX Security Symposium (USENIX Security 18)*. 955–972.
- [9] Daniel Gruss, Clémentine Maurice, Anders Fogh, Moritz Lipp, and Stefan Mangard. 2016. Prefetch Side-Channel Attacks: Bypassing SMAP and Kernel ASLR. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. ACM, Vienna Austria, 368–379. <https://doi.org/10.1145/2976749.2978356>
- [10] Lorenz Hetterich and Michael Schwarz. 2022. Branch different-spectre attacks on apple silicon. In *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, 116–135.
- [11] R. Hund, C. Willems, and T. Holz. 2013. Practical Timing Side Channel Attacks against Kernel Space ASLR. In *2013 IEEE Symposium on Security and Privacy*. IEEE, Berkeley, CA, 191–205. <https://doi.org/10.1109/SP.2013.23>
- [12] Intel. [n. d.]. Running Average Power Limit Energy Reporting. <https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/advisory-guidance/running-average-power-limit-energy-reporting.html>. Accessed: August 2025.
- [13] Hyerean Jang, Taehun Kim, and Youngjoo Shin. 2024. SysBumps: Exploiting Speculative Execution in System Calls for Breaking KASLR in macOS for Apple Silicon. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. ACM, Salt Lake City UT USA, 64–78. <https://doi.org/10.1145/3658644.3690189>
- [14] Yeongjin Jang, Sangho Lee, and Taesoo Kim. 2016. Breaking Kernel Address Space Layout Randomization with Intel TSX. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*. ACM, Vienna Austria, 380–392. <https://doi.org/10.1145/2976749.2978321>
- [15] The kernel development community. [n. d.]. TSX Async Abort (TAA) mitigation. https://docs.kernel.org/arch/x86/tsx_async_abort.html. Accessed: August 2025.
- [16] Suryeon Kim, Seungwon Shin, and Hyunwoo Choi. 2023. AVX-TSCHA: Leaking Information through AVX Extensions in Commercial Processors. *Computers & Security* 134 (2023), 103437. <https://doi.org/10.1016/j.cose.2023.103437>
- [17] Jakob Koschel, Cristiano Giuffrida, Herbert Bos, and Kaveh Razavi. 2020. Tag-Bleed: Breaking KASLR on the Isolated Kernel Address Space Using Tagged TLBs. In *2020 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, Genoa, Italy, 309–321. <https://doi.org/10.1109/EuroSP48549.2020.00027>
- [18] Red Hat Enterprise Linux. [n. d.]. Important changes to external kernel parameters. https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/8/html/8.9_release_notes/kernel_parameters_changes. Accessed: August 2025.
- [19] Moritz Lipp, Daniel Gruss, and Michael Schwarz. 2022. {AMD} prefetch attacks through power and time. In *31st USENIX Security Symposium (USENIX Security 22)*. 643–660.
- [20] Moritz Lipp, Andreas Kogler, David Oswald, Michael Schwarz, Catherine Easdon, Claudio Canella, and Daniel Gruss. 2021. PLATYPUS: Software-based Power Side-Channel Attacks on X86. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, San Francisco, CA, USA, 355–371. <https://doi.org/10.1109/SP40001.2021.00063>
- [21] William Liu, Joseph Ravichandran, and Mengjia Yan. 2023. EntryBleed: A Universal KASLR Bypass against KPTI on Linux. In *Proceedings of the 12th International Workshop on Hardware and Architectural Support for Security and Privacy*. ACM, Toronto Canada, 10–18. <https://doi.org/10.1145/3623652.3623669>
- [22] Ryan Roemer, Erik Buchanan, Hovav Shacham, and Stefan Savage. 2012. Return-oriented programming: Systems, languages, and applications. *ACM Transactions on Information and System Security (TISSEC)* 15, 1 (2012), 1–34.
- [23] Milad Seddigh, Mahdi Esfahani, Sarani Bhattacharya, Mohammad Reza Aref, and Hadi Soleimany. 2022. Breaking KASLR on Mobile Devices without Any Use of Cache Memory. In *Proceedings of the 2022 Workshop on Attacks and Solutions in Hardware Security*. ACM, Los Angeles CA USA, 45–54. <https://doi.org/10.1145/3560834.3563823>
- [24] Zhi Wang and Xuxian Jiang. 2010. Hypersafe: A lightweight approach to provide lifetime hypervisor control-flow integrity. In *2010 IEEE symposium on security and privacy*. IEEE, 380–395.
- [25] Daniel Weber, Ahmad Ibrahim, Hamed Nemati, Michael Schwarz, and Christian Rossow. 2021. Osiris: Automated discovery of microarchitectural side channels. In *30th USENIX Security Symposium (USENIX Security 21)*. 1415–1432.
- [26] YaoYuan. 2024. XNU kperf/kpc demo. <https://gist.github.com/ibireme/173517c208c7dc333ba962c1f0d67d12>.